Characterization of Human Delay Response

Daniel Kadyrov

December 23, 2015

Contents

1 Introduction **2**

2 Discussion **3**
2.1 Case 1 4
2.2 Case 2 8
2.3 Case 3 12
2.4 Case 4 16

3 Summary and Conclusion **20**

4 Appendix **21**
4.1 Data 21
4.2 MATLAB Code 21
4.2.1 Functions 21
4.3 General MATLAB Code 22

List of Figures

1 System Block Diagram 3
2 Practice Results (Case 1) 4
3 Time History (Case 1) 4
4 Power Spectral Density (Case 1) 5
5 Experimental Bode Diagram (Case 1) 6
6 Simulated Bode Diagram (Case 1) 7
7 Practice Results (Case 2) 8
8 Time History (Case 2) 8
9 Power Spectral Density (Case 2) 9
10 Experimental Bode Diagram (Case 2) 10
11 Simulated Bode Diagram (Case 2) 11
12 Practice Results (Case 3) 12
13 Time History (Case 3) 12
14 Power Spectral Density (Case 3) 13
15 Experimental Bode Diagram (Case 3) 14
16 Simulated Bode Diagram (Case 3) 15
17 Practice Results (Case 4) 16
18 Time History (Case 4) 16
19 Power Spectral Density (Case 4) 17
20 Experimental Bode Diagram (Case 4) 18
21 Simulated Bode Diagram (Case 4) 19

1 Introduction

The software package "Man-Machines System Laboratory" allows the measurement of human reaction through a task where a user is asked to keep a pointer within a moving box using a computer mouse as a controller. As the box is continuously moving in a random, non-repeating motion, the user attempts to reject the disturbance and have the pointer remain in the area. Preferences within this application allow changes with the amount of gain at which the mouse movement affects the movement of the pointer as well as with the amount of delay the pointer movement occurs after the mouse has been moved. This generates data on how a human operator is affected by the gains and delays within the system they are using. This data, once modeled, provides insight on the human operator as a mathematical model which can then be adapted on more complicated machines and simulators.

This assignment used the software package in order to measure the human performance in a single degree of freedom, disturbance rejection, task. The parameters changed within the four cases of the experiment were the mouse gain and the delay of the input. Table 1 shows the four cases and the values for the parameters used in their respective cases.

Parameters	**Case 1**	**Case 2**	**Case 2**	**Case 2**
Mouse/Joystick Gain	0.5	0.5	1.0	1.0
Input Time Delay	0	0.15	0	0.15

Table 1: Test Cases

The experiment duration was set to be 60 seconds long per round and the system frequency was set to be at 50 Hz. This meant that the software recorded data every 0.02 seconds, or 20 milliseconds. The target size was set to be at 0.1 and the system dynamics was kept at $\frac{1}{s}$. This function for system dynamics will appear within the block diagram in the Discussion section of this report.

For each case, the user played several practice rounds until reaching an asymptote of peak ability determined by the time, or percentage, on target. The plots for each case are shown within the Discussion section of this report under their respective case subsections. At the asymptotic point, the raw data was extracted from the software in the form of a MATLAB file.

The raw data was used in order to generate a time history graph, bode diagram and power spectral density plot of the system for each case. The MATLAB **System Identification Toolbox** was then used to develop a mathematical model of the human operator. The simulated transfer function was then compared to the experimental data in the frequency domain through bode diagrams.

2 Discussion

A block diagram of the system is shown in Figure 1 below:

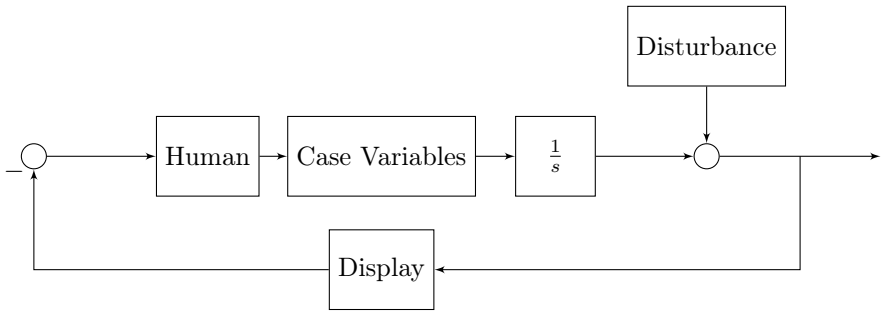


Figure 1: System Block Diagram

In this diagram, "Human" represents the human operator model found in each case. The "Case Variables" block represents the input delay with gain, set in each case, and the control inceptor dynamics set by the program, $\frac{40}{s+40}$. The controlled plant dynamics is also set by the program to be $\frac{1}{s}$.

The "Case Variables" block includes the gain and delay within one function. This function shown below:

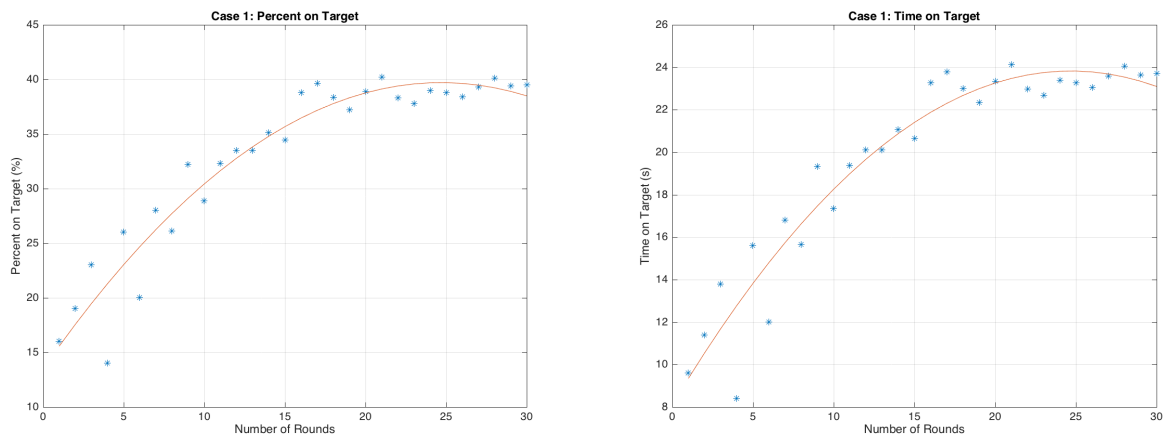
$$D(s) = Ke^{-\tau s} \tag{1}$$

In this equation, K is the gain of the mouse set for each case. The delay amount, τ, is also set for each case.

Within this diagram, the analysis was mainly done on the human operator model block. The input into the block was generated as the "error" data signal. The output of the block was generated as the "input" data signal.

2.1 Case 1

In the first case the mouse gain was set to 0.5 and the input time delay was set to 0. The practice results are shown in Figure 2 (a) and (b).



(a) Percent on Target

(b) Time on Target

Figure 2: Practice Results (Case 1)

The round used for analysis had the user 38.8%, or 23.28 seconds on target. A time history of this case and round is shown in Figure 3.

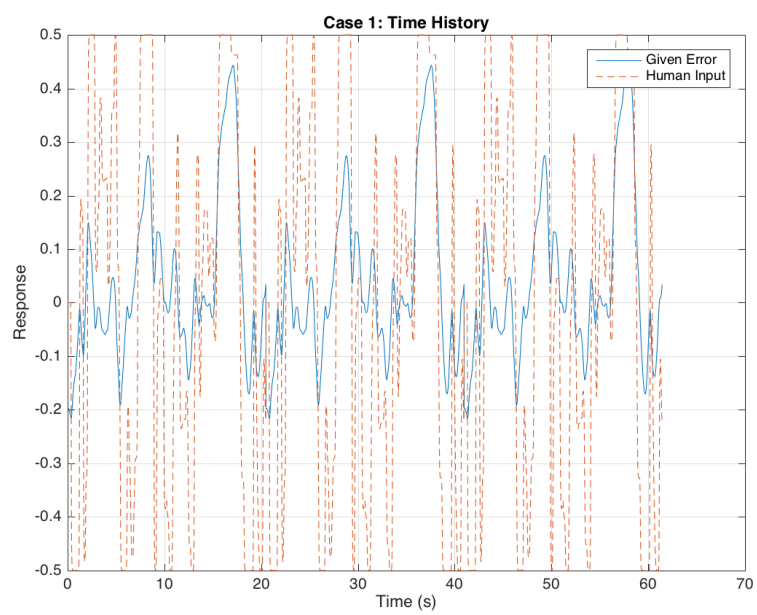


Figure 3: Time History (Case 1)

The function, **PSD**, outputs the power spectral density and frequency domain from an inputted set of signals. It achieves this through buffering, a hann window and fast Fourier transform. This is done through the built in MATLAB functions **buffer**, **hann**, and **fft**, respectively. Besides the signal arrays, the **PSD** function also requires the input of the sampling frequency as well as the segment and overlap size for the buffering command. Varying these values allows for adjustment in order to clean the output. The values in this case were chosen to be 200 and 50, respectively. The power spectral density is shown in Figure 4.

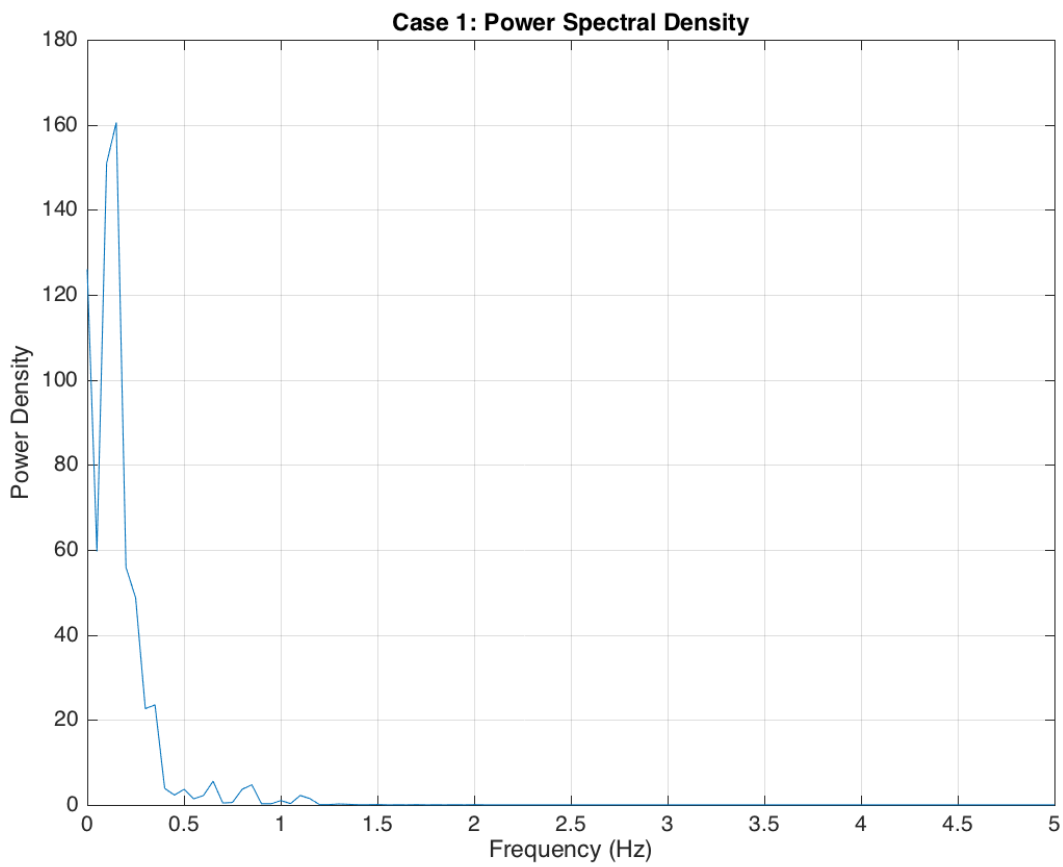


Figure 4: Power Spectral Density (Case 1)

It can be seen that the dominant frequency is at .15 Hz. The function, **PSD**, can be found in the Appendix.

The function, **FRF**, outputs the magnitude, phase, and frequency domain of an inputted set of signals. Like the **PSD** function, it uses buffering, a hann window and fast Fourier transform as well as crosscorrelation and autocorrelation. The MATLAB function for crosscorrelation and autocorrelation is **xcorr**. The values for the segment and buffer size were kept as 1000 and 750, respectively. The bode function for this case is shown in Figure 5.

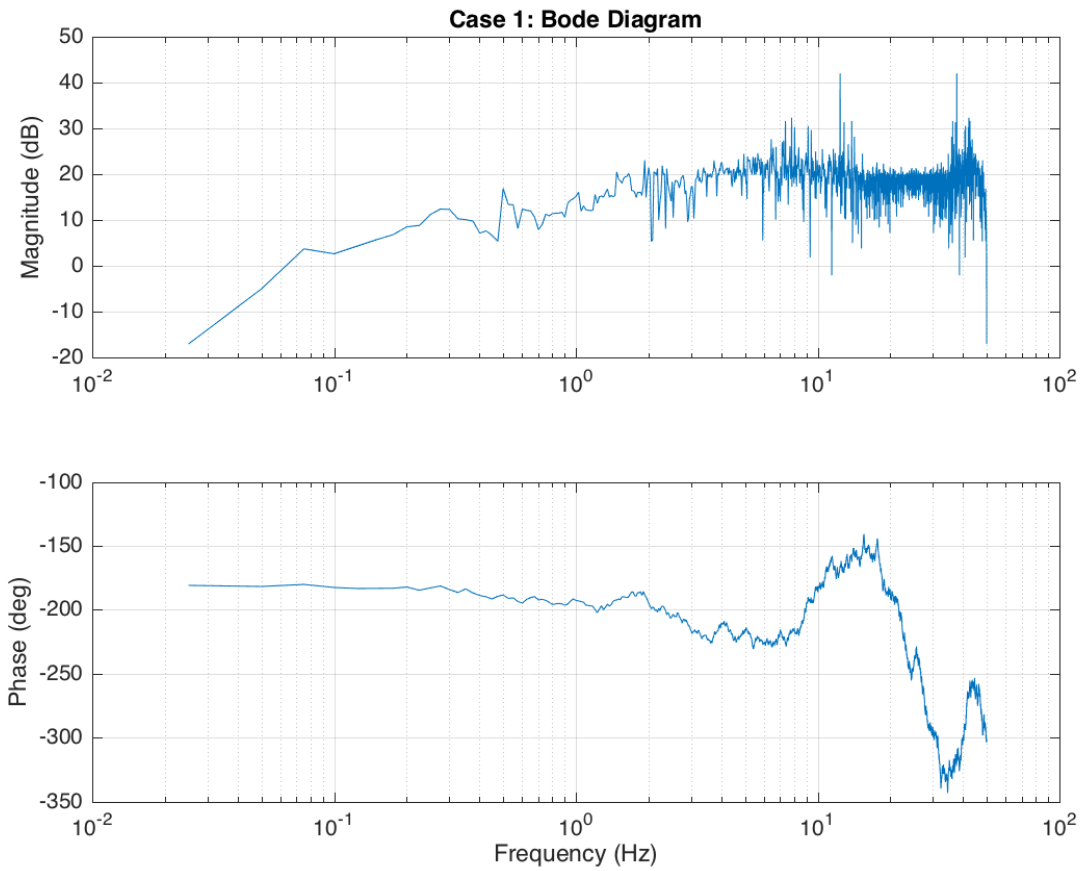


Figure 5: Experimental Bode Diagram (Case 1)

The MATLAB **System Identification Toolbox** is able generate a transfer function from an inputted set of data signals. The data for the error and input were sent through this tool in order to find a transfer function for the human operator. The method used was the autoregressive exogenous (ARX). The transfer function for the human operator model in case 1 is listed below:

$$H(s) = \frac{1.64s^{-2} - 3.06s^{-3} + 1.69s^{-4} - 0.273s^{-5}}{1 - 2.88s^{-1} + 3.26s^{-2} - 1.78s^{-3} + 0.409s^{-4}} \tag{2}$$

This model was then placed within the system loop alongside the input delay and gain, control inceptor dynamics and the controlled plant dynamics. The bode diagram of the total system is shown on the following page.

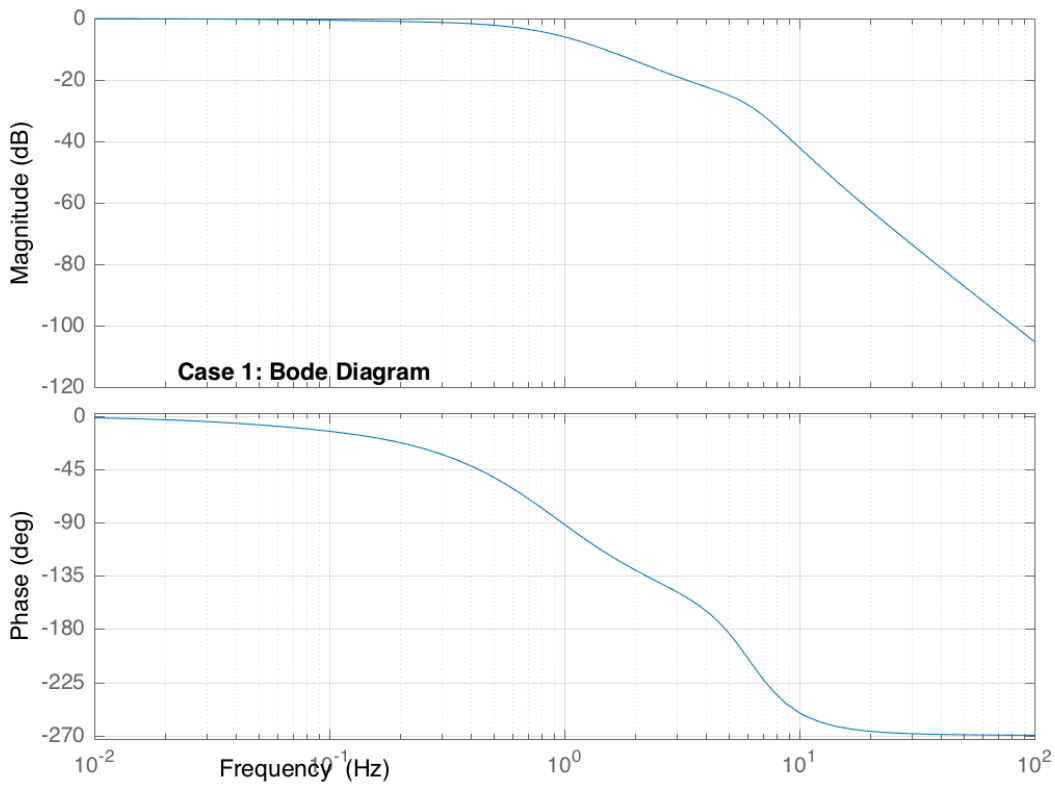


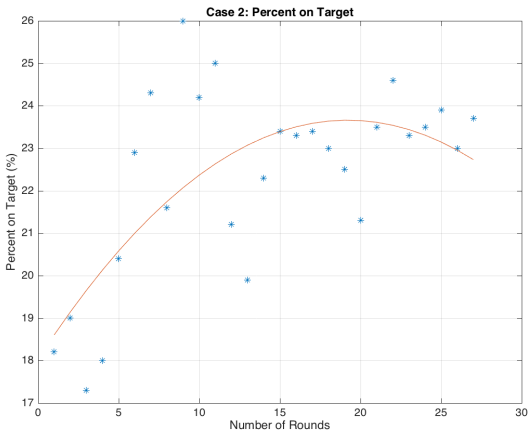
Figure 6: Simulated Bode Diagram (Case 1)

There are some similarities between the frequency response function found from the data and the bode function found through the simulated function. Although the magnitudes are very different, the shape of the phase plots are similar.

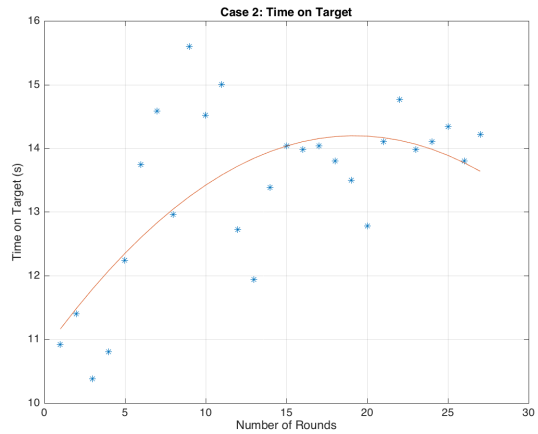
The bode diagram of the simulated model shows that the human operator acts a low pass filter.

2.2 Case 2

In the second case the mouse gain was set to 0.5 and the input time delay was set to .15. The practice results are shown in Figure 7 (a) and (b).



(a) Percent on Target



(b) Time on Target

Figure 7: Practice Results (Case 2)

As seen by the practice results, the difficulty of the task is much higher than in the first case. The user, however, is more familiar with the task so the practice results do not follow as much of an asymptotic curve as the one for previous case. The round used for analysis had the user 23.3%, or 14 seconds on target.
A time history of this case and round is shown in Figure 8.

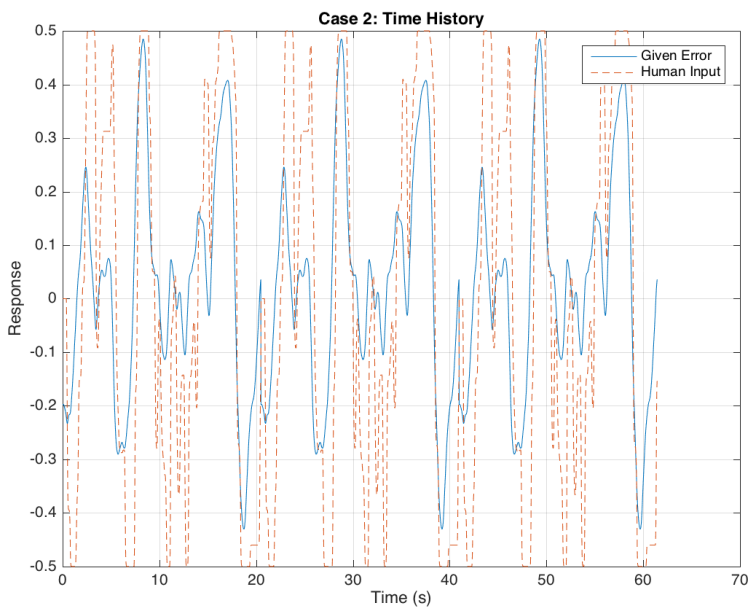


Figure 8: Time History (Case 2)

The power spectral density for case 2 is shown in Figure 9.

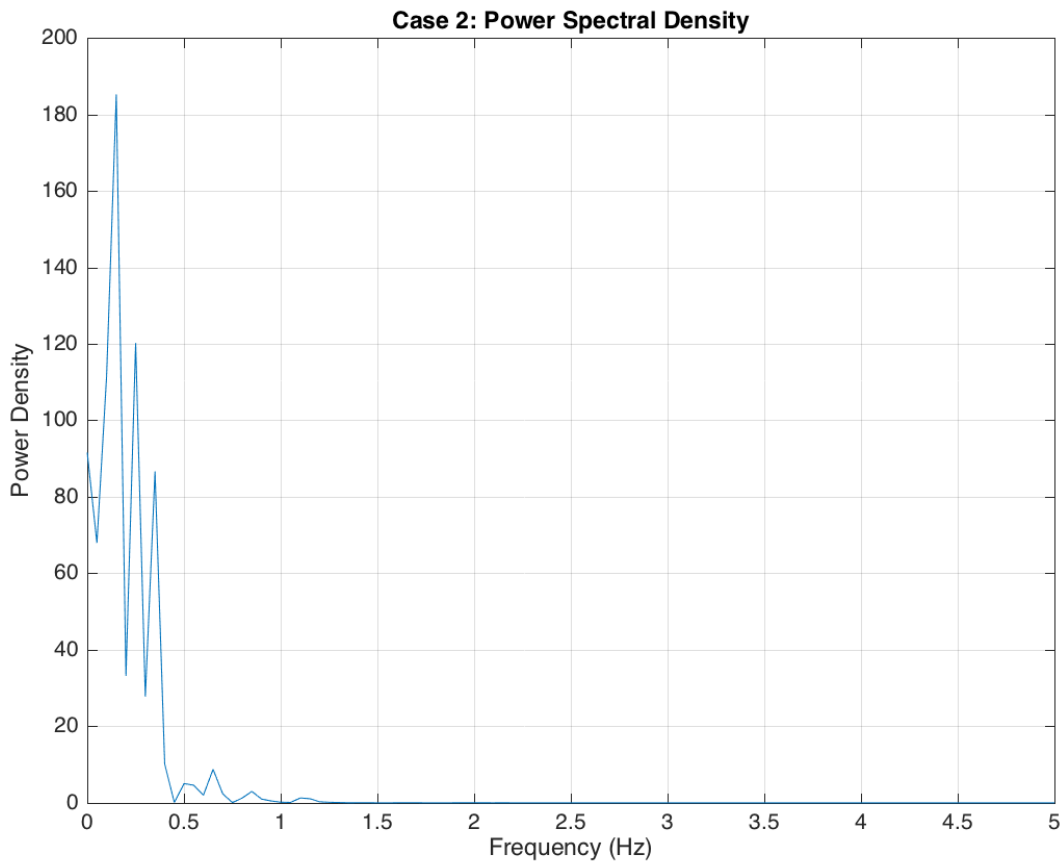


Figure 9: Power Spectral Density (Case 2)

It can be seen that the dominant frequencies are at 0.15 Hz, .25 Hz and .35 Hz.

The bode function for case 2 is shown in Figure 10.

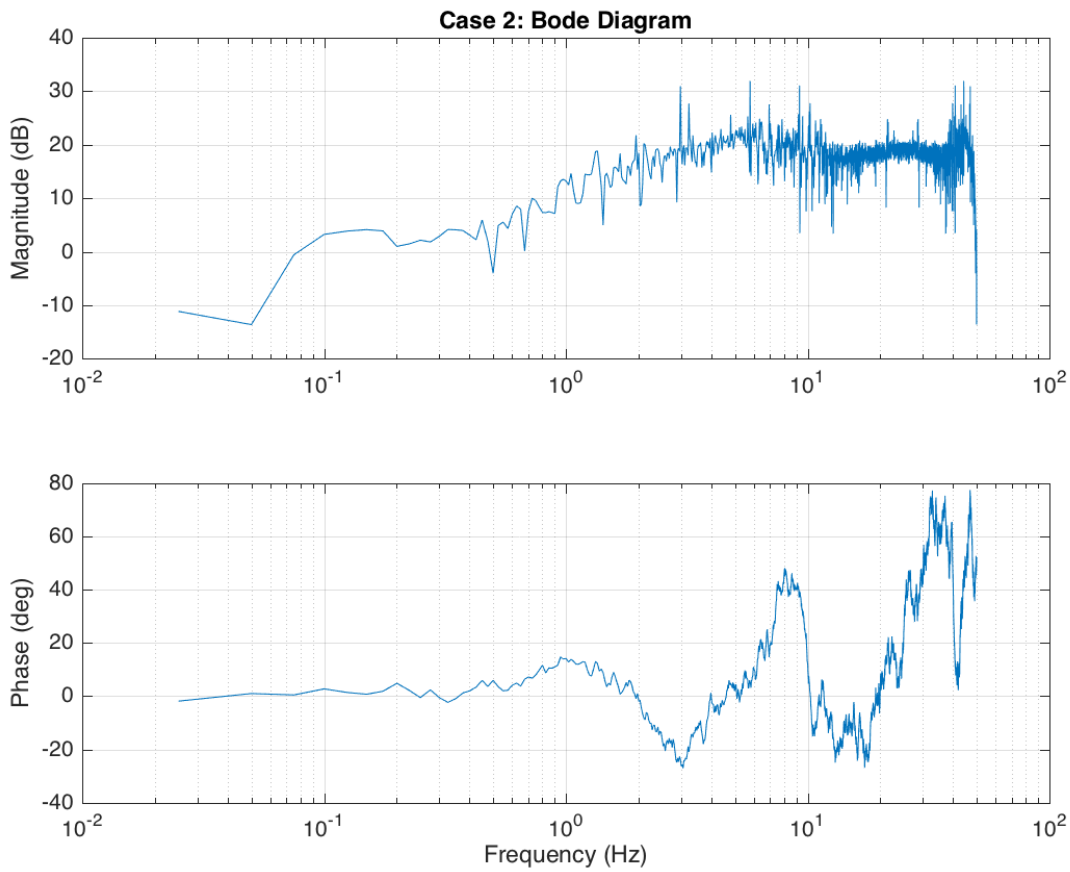


Figure 10: Experimental Bode Diagram (Case 2)

For case 2, the MATLAB **System Identification Toolbox** generated the following transfer function for the human operator.

$$H(s) = \frac{1.66s^{-2} - 2.28s^{-3} + 1.30s^{-4} - 0.19s^{-5}}{1 - 3s^{-1} + 3.55s^{-2} - 2s^{-3} + 0.46s^{-4}} \tag{3}$$

This model was then placed within the system loop alongside the input delay and gain, control inceptor dynamics and the controlled plant dynamics. The bode diagram of the total system is on the following page.

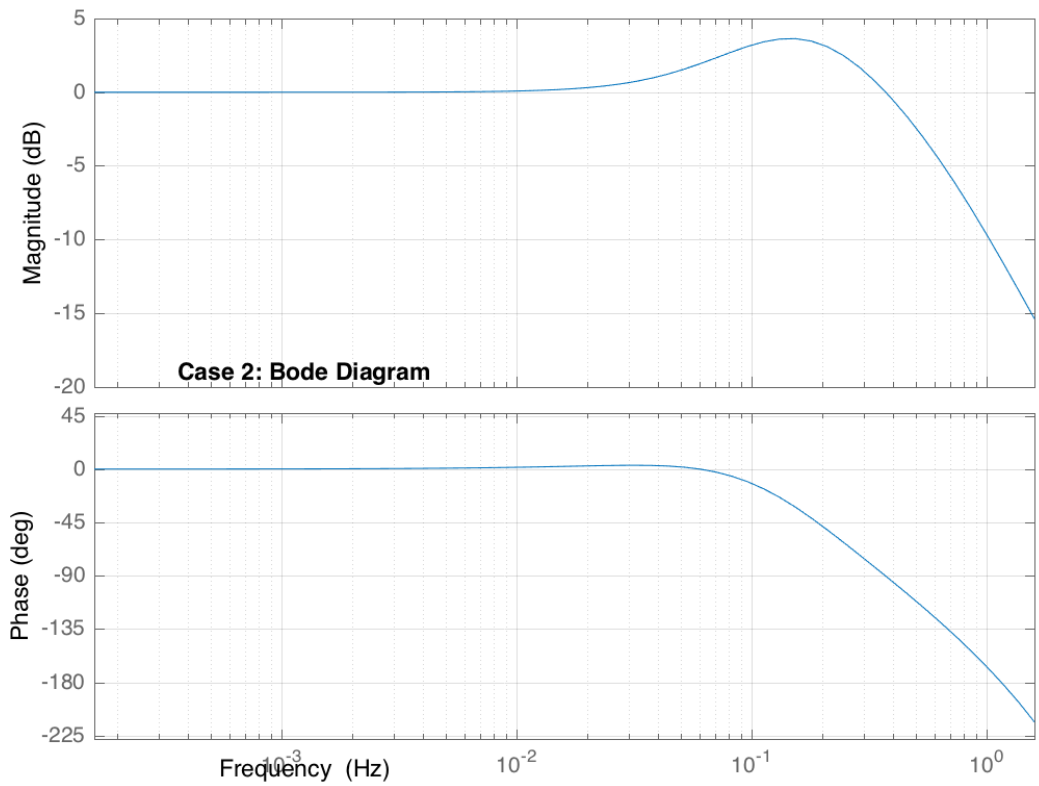
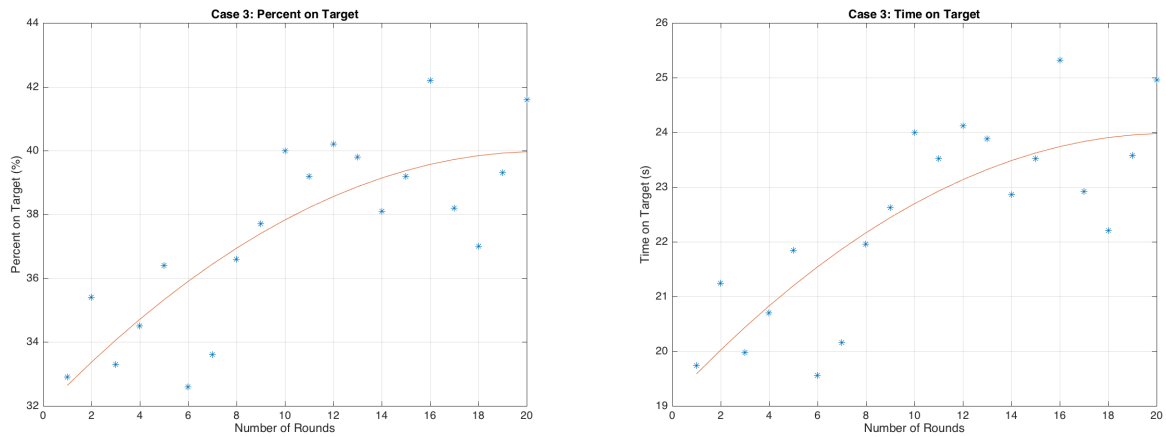


Figure 11: Simulated Bode Diagram (Case 2)

The bode diagram of the simulated model has no similarity to the frequency response function calculated using the data. Like in case 1, the bode diagram for the simulated model shows that the model is a low pass filter. In this case, there is a resonant peak at around 0.25 Hz. This is in correlation with the PSD of this case.

2.3 Case 3

In the third case the mouse gain was set to 1 and the input time delay was set to 0. The practice results are shown in Figure 7 (a) and (b).



(a) Percent on Target

(b) Time on Target

Figure 12: Practice Results (Case 3)

As seen by the practice results, the larger gain and low delay resulted in a larger percent and time on target. Although a period of recalibration was necessary, the user asymptote to around 41.7% or 25 seconds on target.
the difficulty of the task is much higher than in the first case. The user, however, is more familiar with the task so the practice results do not follow as much of an asymptotic curve as the one for previous case. The round used for analysis had the user 23.3%, or 7 seconds on target.
A time history of this case and round is shown in Figure 13.

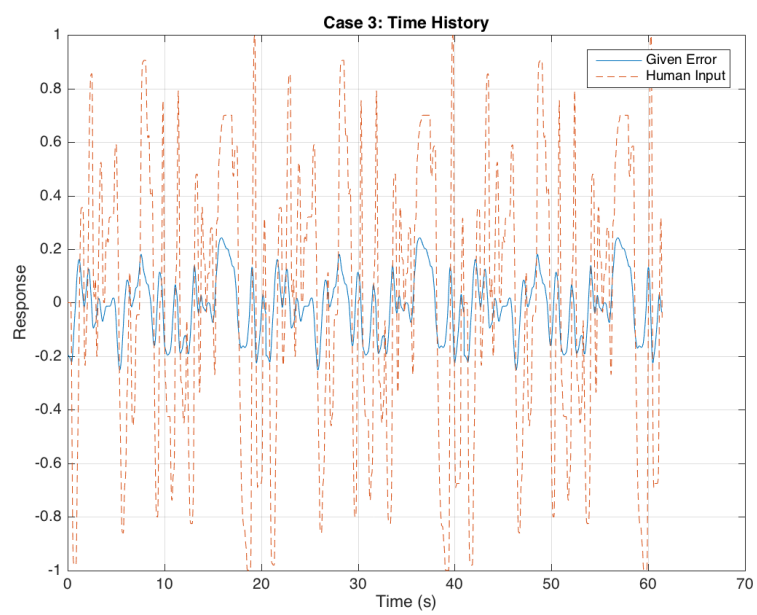


Figure 13: Time History (Case 3)

The power spectral density for case 3 is shown in Figure 14.

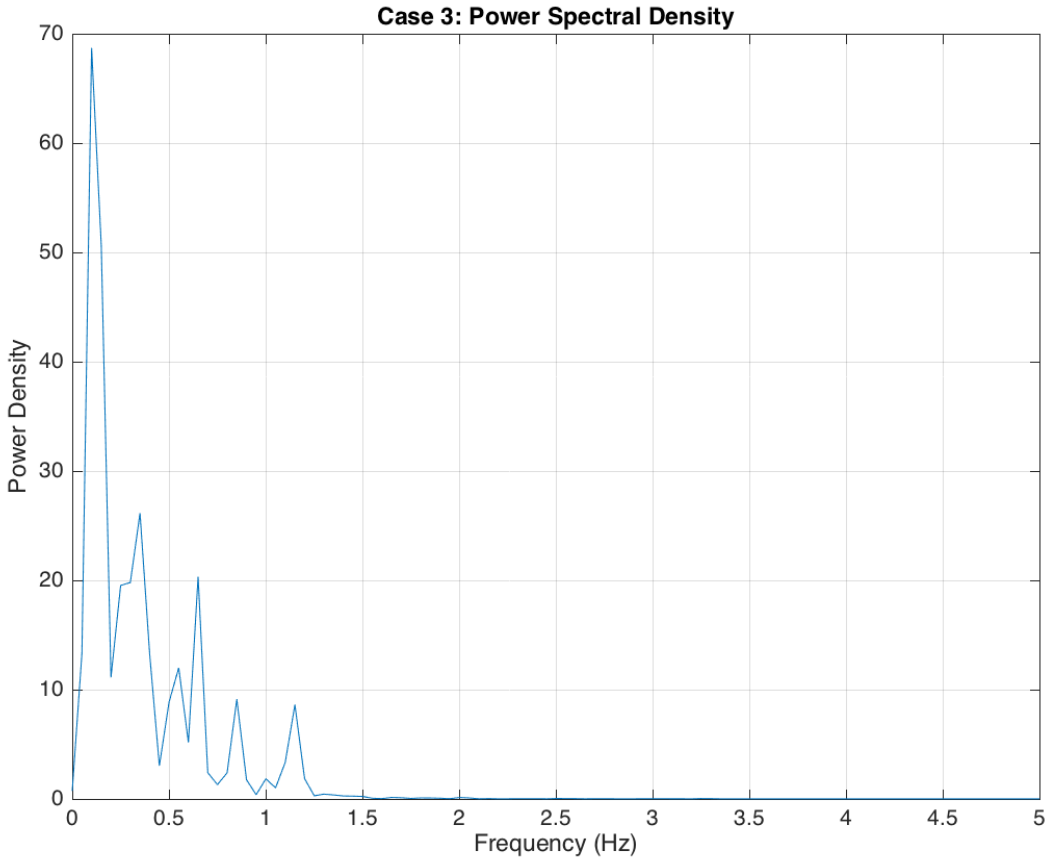


Figure 14: Power Spectral Density (Case 3)

It can be seen that the dominant frequencies are at 0.1 Hz, 0.35 Hz, 0.65 Hz, 0.85 Hz and 1.151 Hz.

The bode function for the data in case 3 is shown in Figure 15.

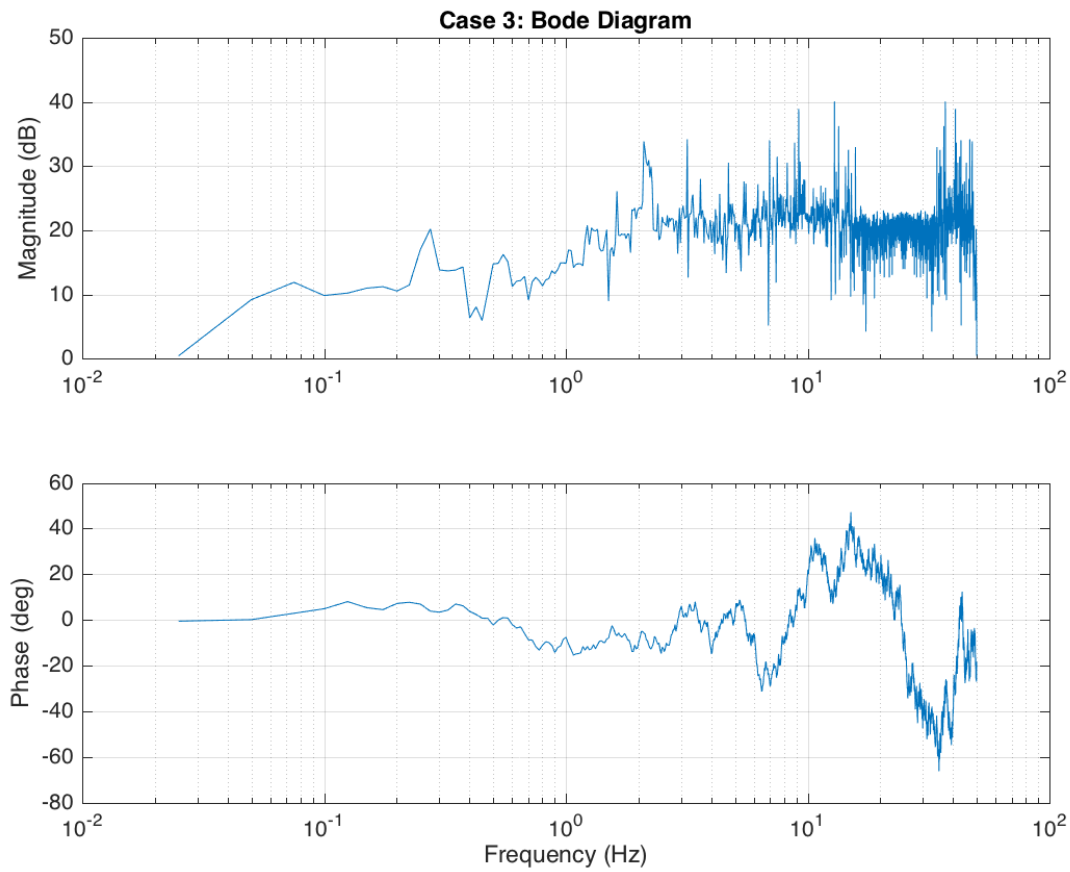


Figure 15: Experimental Bode Diagram (Case 3)

For case 3, the MATLAB **System Identification Toolbox** generated the following transfer function for the human operator.

$$H(s) = \frac{1.03s^{-2} - 1.83s^{-3} + 0.88s^{-4} - 0.07s^{-5}}{1 - 3s^{-1} + 3.51s^{-2} - 1.95s^{-3} + 0.44s^{-4}} \tag{4}$$

This model was then placed within the system loop alongside the input delay and gain, control inceptor dynamics and the controlled plant dynamics. The bode diagram of the total system is shown on the following page.

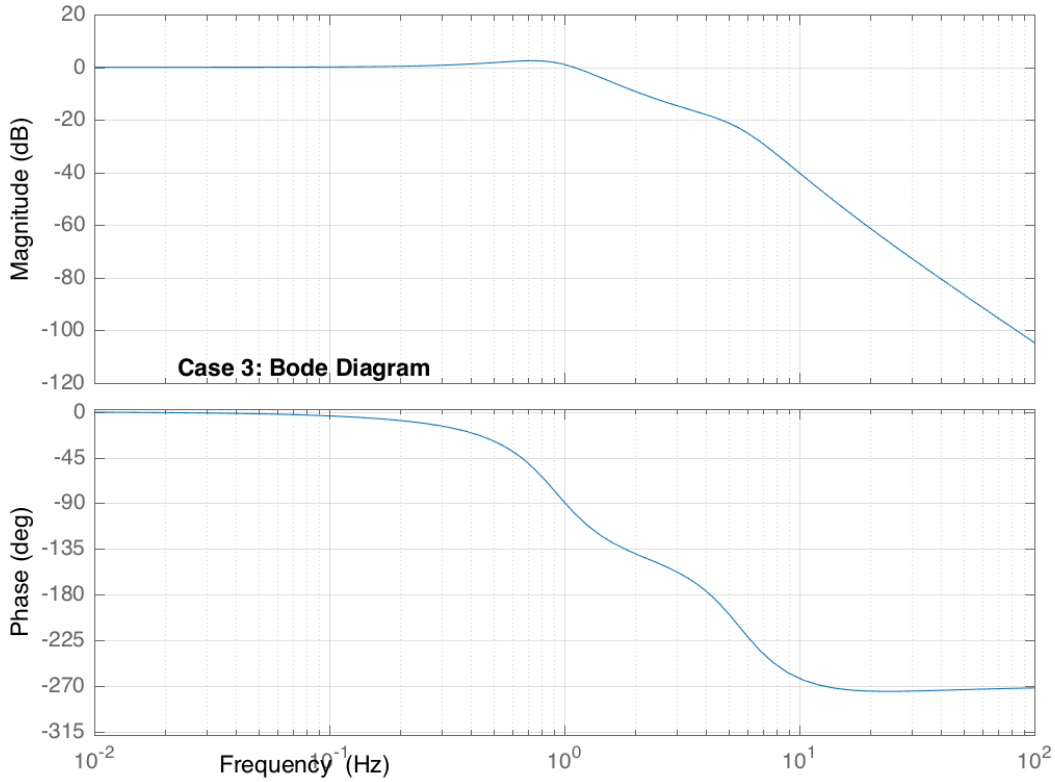
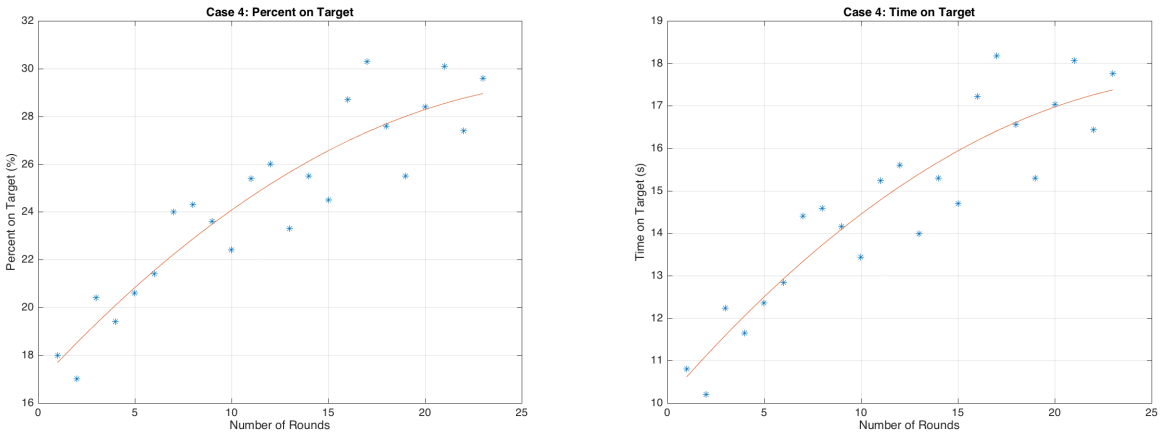


Figure 16: Simulated Bode Diagram (Case 3)

Within this case, the simulated frequency response did not correlate to that of the bode diagram computed from the data results. Like in the previous cases, the simulated frequency response plot shows that the model is a low pass filter. The cutoff frequency in this case starts around 1 Hz.

2.4 Case 4

In the second case the mouse gain was set to 1 and the input time delay was set to 0.15. The practice results are shown in Figure 17 (a) and (b).



(a) Percent on Target

(b) Time on Target

Figure 17: Practice Results (Case 4)

This case proved to be the most difficult one for the user to play. Again, the user had to recalibrate in order to adjust to the new delay on the system. The user asymptotes at 29.6%, or 17.76, seconds on target.

A time history of this case and round is shown in Figure 18.

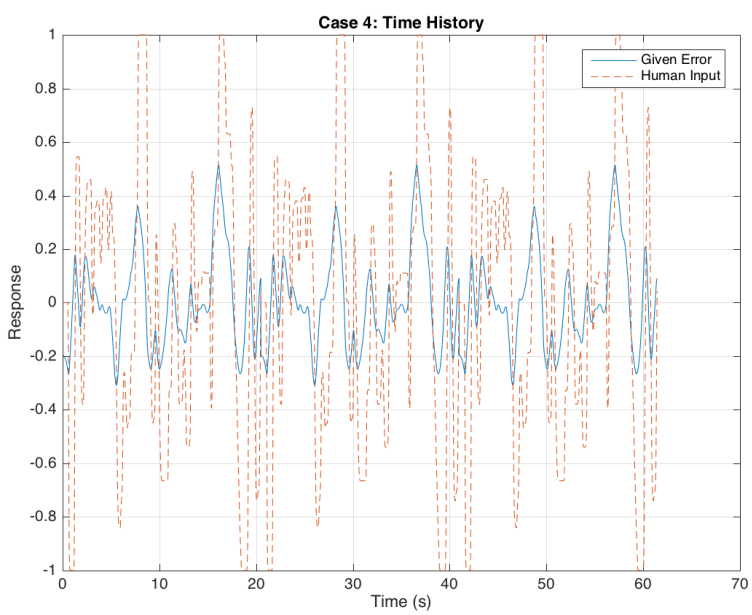


Figure 18: Time History (Case 4)

The power spectral density of case 4 is shown in Figure 19.

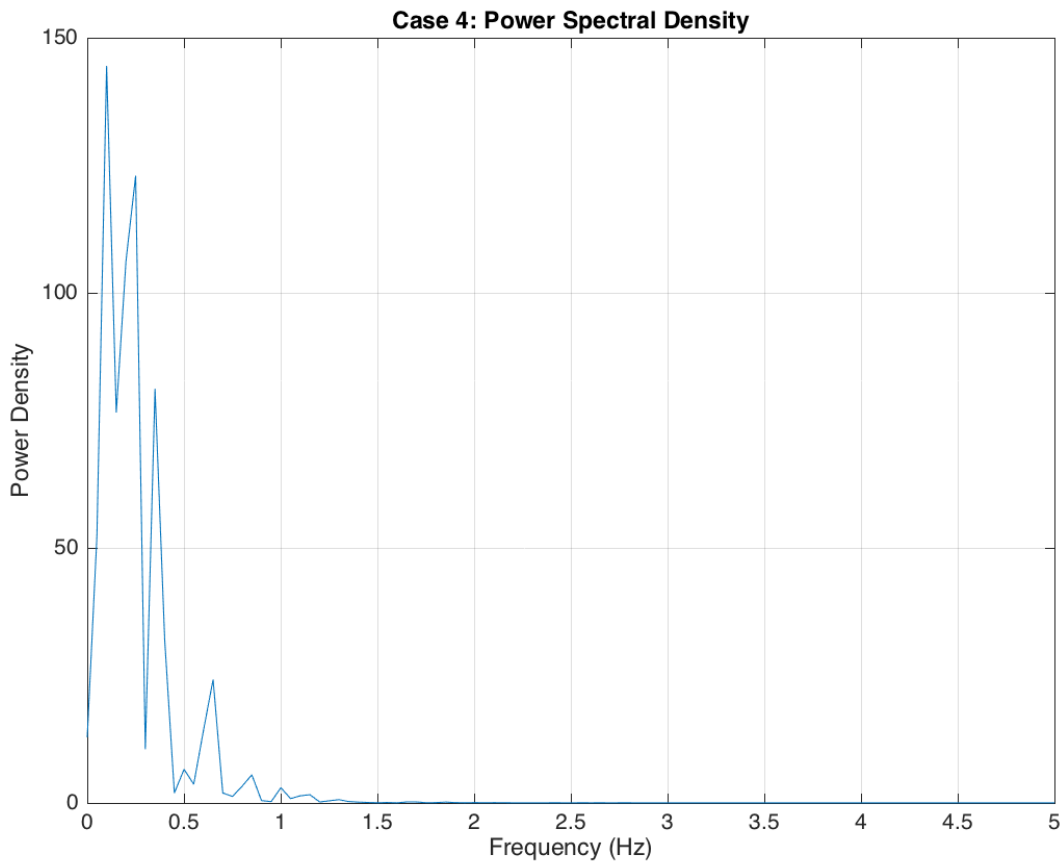


Figure 19: Power Spectral Density (Case 4)

It can be seen that the dominant frequencies are at 0.1 Hz, 0.25 Hz, 0.35 Hz and 0.65 Hz.

The bode function for case 4 is shown in Figure 20.

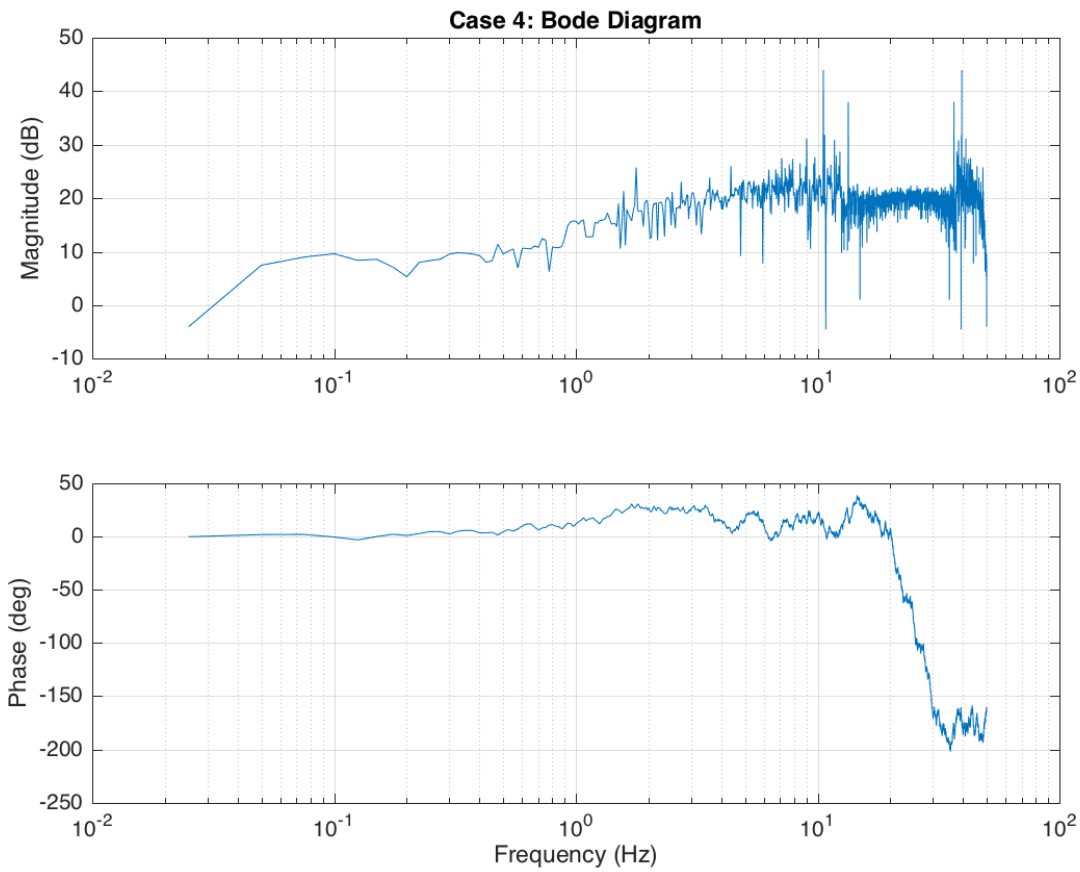


Figure 20: Experimental Bode Diagram (Case 4)

For case 4, the MATLAB **System Identification Toolbox** generated the following transfer function for the human operator.

$$H(s) = \frac{-0.5s^{-2} - 0.90s^{-3} + 0.38s^{-4} - 002s^{-5}}{1 - 2.97s^{-1} + 3.42s^{-2} - 1.82s^{-3} + 0.383s^{-4}} \tag{5}$$

This model was then placed within the system loop alongside the input delay and gain, control inceptor dynamics and the controlled plant dynamics.. The bode diagram of the total system is shown below:

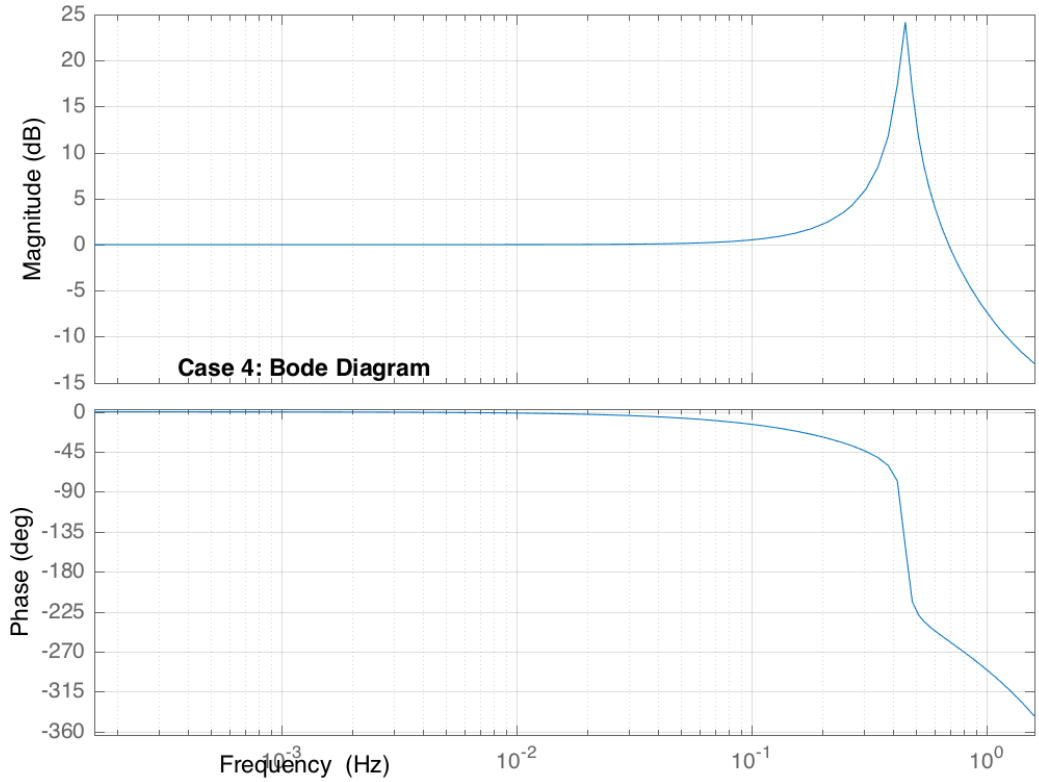


Figure 21: Simulated Bode Diagram (Case 4)

Unlike the previous cases, the frequency response plots of the data and the simulated model look similar. Although the shape of the magnitude plots are slightly different, they both reach an average maximum magnitude of around 25 dB. The phase plots meanwhile follow the exact same shape, however their range difference is off by approximately 100 degrees.

3 Summary and Conclusion

The software package "Man-Machines System Laboratory" was used in order to determine a mathematical model of a human operator. Various conditions were changed and analyzed. These conditions including the time on the input delay and the mouse gain.

The data from these runs was exported into MATLAB and the time history, power spectral density, and frequency response function plots were calculated. This data was then used within the MATLAB **System Identification Toolbox** in order to find a mathematical model for the human operator using ARX. This transfer function was inserted into the loop of the system and the frequency response of the simulated system was generated. The frequency response function of the simulated system was then compared to the one calculated through the data.

The frequency response plots were exceptionally different from each other. This can be due to a variety of reasons. Firstly, the data was measured using a mouse instead of a joystick. A mouse derives its motion based on its current location while a joystick returns to an initial condition. Using a joystick would not only allow for a larger on-target amount, it will also provide better quantitative results. Next, the time period of the data collection was set to 60 seconds with a sampling frequency of 50 Hz. With a larger sampling frequency, like 100-200 Hz, and a longer time period, more data could be generated and allow for better plots. Finally the functions used to find the frequency response function and power spectral density are much similar than the algorithms used by MATLAB's **System Identification Toolbox**. For this reason, the mathematical models generated by MATLAB are correct representations of the human operator model.

4 Appendix

All files displayed within the Appendix section are available in the "Appendix" folder part of this file.

4.1 Data

The raw data files are provided in the folder marked "Appendix" within this file.

4.2 MATLAB Code

4.2.1 Functions

```
funtion [PSD,f] = PSD(Input,Output,Fs,N,P)
  I = buffer(Input,N,P,'nodelay'); %break input into segments
  O = buffer(Output,N,P,'nodelay');%break output into segments
  f=0:Fs/(N-1):Fs; % Creates Frequency Domain with 4000 Points
  win_hann = hann(N,'periodic');%create Hann window
    for ii = size(I,2);
        winX(:,ii)=win_hann.*I(:,ii); % Apply Hann window
        Xfft(:,ii)=fft(winX(:,ii)); % FFT
        S(:,ii)=Xfft(:,ii).*conj(Xfft(:,ii)); % Multiplies fft by conjugate
    end
  PSD=mean(abs(S),2);
end
```

Code Excerpt 1: Power Spectral Density

```
function [f,Magabs,Magdb,Ph] = FRF(Input,Output,Fs,N,P)
  I = buffer(Input,N,P,'nodelay'); %break input into segments
  O = buffer(Output,N,P,'nodelay');%break output into segments

  win_hann = hann(N,'periodic');%create Hann window

  f = 0:Fs/(2*(N-1)):Fs; % Creates a new frequency domain with 7999 points.

  for ii = 1:size(I,2);
      wInput(:,ii) = win_hann.*I(:,ii); % Apply hann window on input
      wOutput(:,ii) = win_hann.*O(:,ii); % Apply hann window on output
      aCorr(:,ii) = fft(xcorr(wInput(:,ii))); % Compute autocorrelation
      Cross(:,ii) = fft(xcorr(wInput(:,ii),wOutput(:,ii))); % Compute cross correlation
       H(:,ii) = Cross(:,ii) ./ aCorr(:,ii); % FRF, cross correlation / autocorrelation
  end
  H=mean(H,2)
  Magabs = abs(H); % Magnitude is absolute value of FRF
  Magdb = 20*log10(Magabs); % Converts magnitude to dB
  Ph = -unwrap(180*angle(H)/pi,-90); % Finds phase angle
end
```

Code Excerpt 2: Frequency Response Function

4.3 General MATLAB Code

```
%% Daniel Kadyrov
% Project 1 - ME 530

clear all
clc

RawData_1=xlsread('Raw_Data.xls','Case 1');
RawData_2=xlsread('Raw_Data.xls','Case 2');
RawData_3=xlsread('Raw_Data.xls','Case 3');
RawData_4=xlsread('Raw_Data.xls','Case 4');

%% Case 1: Target
PercentTarget=[16 19 23 14 26 20 28 26.1 32.2 28.9 32.3 33.5 33.5 35.13 34.43 38.8 ...
        39.64 38.34 37.23 38.9 40.2 38.3 37.8 39 38.8 38.4 39.3 40.1 39.4 39.5];
x=1:length(PercentTarget);
p = polyfit(x,PercentTarget,2);
y1 = polyval(p,x);
figure(1)
plot(x,PercentTarget,'*',x,y1)
grid on
title('Case 1: Percent on Target')
xlabel('Number of Rounds')
ylabel('Percent on Target (%)')

timeTarget=PercentTarget/100*60;
t = polyfit(x,timeTarget,2);
y2 = polyval(t,x);
figure(2)
plot(x,timeTarget,'*',x,y2)
grid on
title('Case 1: Time on Target')
xlabel('Number of Rounds')
ylabel('Time on Target (s)')

%% Case 2: Target
PercentTarget=[18.2 19 17.3 18 20.4 22.9 24.3 21.6 26 24.2 25.0 21.2 19.9 22.3 ...
                23.4 23.3 23.4 23.0 22.5 21.3 23.5 24.6 23.3 23.5 23.9 23 23.7];
x=1:length(PercentTarget);
p = polyfit(x,PercentTarget,2);
y1 = polyval(p,x);
figure(3)
plot(x,PercentTarget,'*',x,y1)
grid on
title('Case 2: Percent on Target')
xlabel('Number of Rounds')
ylabel('Percent on Target (%)')

timeTarget=PercentTarget/100*60;
```

```
t = polyfit(x,timeTarget,2);
y2 = polyval(t,x);
figure(4)
plot(x,timeTarget,'*',x,y2)
grid on
title('Case 2: Time on Target')
xlabel('Number of Rounds')
ylabel('Time on Target (s)')

%% Case 3: Target
PercentTarget=[32.9 35.4 33.3 34.5 36.4 32.6 33.6 36.6 37.7 40 ...
               39.2 40.2 39.8 38.1 39.2 42.2 38.2 37 39.3 41.6];
x=1:length(PercentTarget);
p = polyfit(x,PercentTarget,2);
y1 = polyval(p,x);
figure(5)
plot(x,PercentTarget,'*',x,y1)
grid on
title('Case 3: Percent on Target')
xlabel('Number of Rounds')
ylabel('Percent on Target (%)')

timeTarget=PercentTarget/100*60;
t = polyfit(x,timeTarget,2);
y2 = polyval(t,x);
figure(6)
plot(x,timeTarget,'*',x,y2)
grid on
title('Case 3: Time on Target')
xlabel('Number of Rounds')
ylabel('Time on Target (s)')

%% Case 4: Target
PercentTarget=[18 17 20.4 19.4 20.6 21.4 24 24.3 23.6 22.4 25.4 26 ...
               23.3 25.5 24.5 28.7 30.3 27.6 25.5 28.4 30.1 27.4 29.6];
x=1:length(PercentTarget);
p = polyfit(x,PercentTarget,2);
y1 = polyval(p,x);
figure(7)
plot(x,PercentTarget,'*',x,y1)
grid on
title('Case 4: Percent on Target')
xlabel('Number of Rounds')
ylabel('Percent on Target (%)')

timeTarget=PercentTarget/100*60;
t = polyfit(x,timeTarget,2);
y2 = polyval(t,x);
figure(8)
plot(x,timeTarget,'*',x,y2)
grid on
title('Case 4: Time on Target')
xlabel('Number of Rounds')
ylabel('Time on Target (s)')
```

```
%% Case 1 FRF/PSD
Input_1=RawData_1(:,6);
Output_1=RawData_1(:,2);

figure(9)
[f,Magabs,Magdb,Ph]=FRF(Input_1,Output_1,50,1000,750);
subplot(2,1,1)
semilogx(f,Magdb)
title('Frequency Response Function')
ylabel('Magnitude (dB)')
grid on
title('Case 1: Bode Diagram')

subplot(2,1,2)
semilogx(f,Ph)
xlabel('Frequency (Hz)')
ylabel('Phase (deg)')
grid on

[PSD1,f1]=PSD(Input_1,Output_1,50,1000,750);
figure(10)
plot(f1,PSD1)
title('Case 1: Power Spectral Density')
xlabel('Frequency (Hz)')
xlim([0 5])
ylabel('Power Density')
grid on

%% Case 2 FRF/PSD
Input_2=RawData_2(:,6);
Output_2=RawData_2(:,2);

figure(11)
[f,Magabs,Magdb,Ph]=FRF(Input_2,Output_2,50,1000,750);
subplot(2,1,1)
semilogx(f,Magdb)
title('Frequency Response Function')
ylabel('Magnitude (dB)')
grid on
title('Case 2: Bode Diagram')

subplot(2,1,2)
semilogx(f,Ph)

xlabel('Frequency (Hz)')
ylabel('Phase (deg)')
grid on

[PSD2,f]=PSD(Input_2,Output_2,50,1000,750);
figure(12)
plot(f,PSD2)
title('Case 2: Power Spectral Density')
xlabel('Frequency (Hz)')
```

```
xlim([0 5])
ylabel('Power Density')
grid on

%% Case 3

Input_3=RawData_3(:,6);
Output_3=RawData_3(:,2);

figure(13)
[f,Magabs,Magdb,Ph]=FRF(Input_3,Output_3,50,1000,750);
subplot(2,1,1)
semilogx(f,Magdb)
title('Frequency Response Function')
ylabel('Magnitude (dB)')
grid on
title('Case 3: Bode Diagram')

subplot(2,1,2)
semilogx(f,Ph)
xlabel('Frequency (Hz)')
ylabel('Phase (deg)')
grid on

[PSD3,f]=PSD(Input_3,Output_3,50,1000,750);
figure(14)
plot(f,PSD3)
title('Case 3: Power Spectral Density')
xlabel('Frequency (Hz)')
xlim([0 5])
ylabel('Power Density')
grid on

%% Case 4

Input_4=RawData_4(:,6);
Output_4=RawData_4(:,2);

figure(15)
[f,Magabs,Magdb,Ph]=FRF(Input_4,Output_4,50,1000,750);
subplot(2,1,1)
semilogx(f,Magdb)
title('Frequency Response Function')
ylabel('Magnitude (dB)')
grid on
title('Case 4: Bode Diagram')

subplot(2,1,2)
semilogx(f,Ph)
xlabel('Frequency (Hz)')
ylabel('Phase (deg)')
grid on

[PSD4,f]=PSD(Input_4,Output_4,50,1000,750);
```

```
figure(16)
plot(f,PSD4)
title('Case 4: Power Spectral Density')
xlabel('Frequency (Hz)')
xlim([0 5])
ylabel('Power Density')
grid on

%% Case 1 Ident
%Using ARX

A=[1,-2.884256937779469,3.261082330549527,
  -1.785263250355974,0.409659142804606];
B=[0,1.643133702652533,-3.061110850606612,
  1.695892615659937,-0.272732637155398];
P = filt(B,A,0.02);
Human = d2c(P,'zoh',0.02)

K=0.5;
Td=0;
s=tf('s')
gain=K;
delay=exp(-Td*s);
filter=40/(40+s);
SD=1/s; %System Dynamics
Loop = Human * gain * delay * filter * SD;
FLoop = feedback(Loop,1)

figure(17)
h = bodeplot(FLoop);
setoptions(h,'FreqUnits','Hz')
grid on
title('Case 1: Bode Diagram')

%figure(16)
%pxx = periodogram(PP)
%plot(pxx)

%% Case 2 Ident
A=[1,-3.008008791631864,3.548505673178966,
  -2.004181379889984,0.463251206968935];
B=[0,1.166404011033924,-2.276888821328781,
  1.303624176088581,-0.191210610936786];

P = filt(B,A,0.02);
Human = d2c(P,'zoh',0.02)

K=0.5;
Td=0.15;
s=tf('s')
gain=K;
delay=exp(-Td*s);
filter=40/(40+s);
SD=1/s; %System Dynamics
```

```
Loop = Human * gain * delay * filter * SD;
FLoop = feedback(Loop,1)

figure(18)
Case2_bode = bodeplot(FLoop, {10^-3, 10^1});
setoptions(Case2_bode,'FreqUnits','Hz')
grid on
title('Case 2: Bode Diagram')

%% Case 3 Ident
A=[1,-3.002466845027131,3.505748460135286,
  -1.946647458149829,0.444981518734658];
B=[0,1.029718675652325,-1.828291453380468,
  0.882133587633281,-0.068808196443920];

P = filt(B,A,0.02);
Human = d2c(P,'zoh',0.02)

K=1;
Td=0;
s=tf('s')
gain=K;
delay=exp(-Td*s);
filter=40/(40+s);
SD=1/s; %System Dynamics
Loop = Human * gain * delay * filter * SD;
FLoop = feedback(Loop,1)

figure(19)
h = bodeplot(FLoop);
setoptions(h,'FreqUnits','Hz')

grid on
title('Case 3: Bode Diagram')

%% Case 4 Indent
A=[1,-2.971388236942425,3.419518177864026,
  -1.824016857536152,0.382508214350125];
B=[0,-0.504748068974189,0.904705369174880,
   -0.380812980955028,-0.002221391147864];
P = filt(B,A,0.02);
Human = d2c(P,'zoh',0.02)

K=1;
Td=0.15;
s=tf('s')
gain=K;
delay=exp(-Td*s);
filter=40/(40+s);
SD=1/s; %System Dynamics
Loop = Human * gain * delay * filter * SD;
FLoop = feedback(Loop,1)

figure(20)
```

```
Case4_bode = bodeplot(FLoop, {10^-3, 10^1});
setoptions(Case4_bode,'FreqUnits','Hz')
grid on
title('Case 4: Bode Diagram')

%% Time Data
time_1=RawData_1(:,1);
time_2=RawData_2(:,1);
time_3=RawData_3(:,1);
time_4=RawData_4(:,1);

figure(21)
plot(time_1,Input_1,time_1,Output_1,'--')
title('Case 1: Time History')
legend('Given Error','Human Input')
ylabel('Response')
xlabel('Time (s)')
grid on

figure(22)
plot(time_2,Input_2,time_2,Output_2,'--')
title('Case 2: Time History')
legend('Given Error','Human Input')
ylabel('Response')
xlabel('Time (s)')
grid on

figure(23)
plot(time_3,Input_3,time_3,Output_3,'--')
title('Case 3: Time History')
legend('Given Error','Human Input')
ylabel('Response')
xlabel('Time (s)')
grid on

figure(24)
plot(time_4,Input_4,time_4,Output_4,'--')
title('Case 4: Time History')
legend('Given Error','Human Input')
ylabel('Response')
xlabel('Time (s)')
grid on
```